新增叢集

Astra Control Service

NetApp
March 27, 2024

This PDF was generated from https://docs.netapp.com/zh-tw/astra-control-service/aws/get-started/install-astra-connector.html on March 27, 2024. Always check docs.netapp.com for the latest.

目錄

將叢集新增至 Astra Control Service 1
安裝 Astra Connector for Private 叢集 1
新增由供應商管理的叢集 4

將叢集新增至 Astra Control Service

設定環境之後、您就可以建立Kubernetes叢集、然後將其新增至Astra Control Service。這可讓您使用 Astra Control Service 來保護叢集上的應用程式。

視您需要新增至 Astra Control Service 的叢集類型而定、您需要使用不同的步驟來新增叢集。

- "將公有供應商託管的叢集新增至 Astra Control Service"：請使用以下步驟來新增具有公有 IP 位址且由雲端供應商管理的叢集。您需要雲端供應商的服務主體帳戶、服務帳戶或使用者帳戶。
- "將私有提供者管理的叢集新增至 Astra Control Service"：請使用以下步驟來新增具有私有 IP 位址且由雲端供應商管理的叢集。您需要雲端供應商的服務主體帳戶、服務帳戶或使用者帳戶。
- "將公用自我管理叢集新增至 Astra Control Service"：請使用以下步驟來新增具有公用 IP 位址且由組織管理的叢集。您需要為想要新增的叢集建立一個 kubeconfig 檔案。
- "將私有自我管理叢集新增至 Astra Control Service"：請使用以下步驟來新增具有私有 IP 位址且由組織管理的叢集。您需要為想要新增的叢集建立一個 kubeconfig 檔案。

安裝 Astra Connector for Private 叢集

Astra Control Service 使用 Astra Connector 來啟用 Astra Control Service 與私有叢集之間的通訊。您需要在要管理的私有叢集上安裝 Astra Connector 。

Astra Connector 支援下列類型的私有叢集：

- Amazon Elastic Kubernetes Service （ EKS ）
- Azure Kubernetes服務（KS）
- Google Kubernetes Engine（GKE）
- AWS 上的 Red Hat OpenShift 服務（ ROSA ）
- ROSA 搭配 AWS Private Link
- 內部部署 Red Hat OpenShift Container Platform

安裝 Astra Connector

關於這項工作

- 當您執行這些步驟時、請針對您要使用 Astra Control Service 管理的私有叢集執行這些命令。
- 如果您使用的是堡壘主機、請從堡壘主機的命令列發出這些命令。

開始之前

- 您需要存取想要使用 Astra Control Service 管理的私有叢集。
- 您需要 Kubernetes 管理員權限、才能在叢集上安裝 Astra Connector 運算子。

步驟

1. 在您要使用 Astra Control Service 管理的私有叢集上安裝 Astra Connector 運算子。當您執行此命令時、命名空間 `astra-connector-operator` 已建立並套用組態至命名空間：

```
kubectl apply -f https://github.com/NetApp/astra-connector-
operator/releases/download/23.07.0-
202310251519/astraconnector_operator.yaml
```

2. 確認操作員已安裝就緒：

```
kubectl get all -n astra-connector-operator
```

3. 從 Astra Control 取得 API 權杖。請參閱 "Astra Automation文件" 以取得相關指示。
4. 建立 Astra 連接器命名空間：

```
kubectl create ns astra-connector
```

5. 建立 Astra Connector CR 檔案並命名 `astra-connector-cr.yaml`。更新括弧 <> 中的值以符合 Astra Control 環境和叢集組態：
   - * <ASTRA_CONTROL_SERVICE_URL> * ： Astra 控制服務的網路 UI URL 。例如：

     ```
     https://astra.netapp.io
     ```

   - * <ASTRA_CONTROL_SERVICE_API_TOKEN> * ：您在上述步驟中取得的 Astra 控制 API 權杖。
   - * <PRIVATE_AKS_CLUSTER_NAME> * ：（僅限 AKS 叢集） - 私有 Azure Kubernetes 服務叢集的叢集名稱。只有在您要新增私有 aks 叢集時、才取消註解並填入此行。
   - * <ASTRA_CONTROL_ACCOUNT_ID> * ：從 Astra Control 網路 UI 取得。選取頁面右上角的圖示、然後選取 * API 存取 * 。

```yaml
apiVersion: netapp.astraconnector.com/v1
kind: AstraConnector
metadata:
  name: astra-connector
  namespace: astra-connector
spec:
  natssync-client:
    cloud-bridge-url: <ASTRA_CONTROL_SERVICE_URL>
  imageRegistry:
    name: theotw
    secret: ""
  astra:
    token: <ASTRA_CONTROL_SERVICE_API_TOKEN>
    #clusterName: <PRIVATE_AKS_CLUSTER_NAME>
    accountId: <ASTRA_CONTROL_ACCOUNT_ID>
    acceptEULA: yes
```

6. 填入之後 `astra-connector-cr.yaml` 使用正確值的檔案、請套用 CR ：

```
kubectl apply -f astra-connector-cr.yaml
```

7. 確認 Astra Connector 已完全部署：

```
kubectl get all -n astra-connector
```

8. 確認叢集已向 Astra Control 註冊：

```
kubectl get astraconnector -n astra-connector
```

您應該會看到類似下列的輸出：

```
NAME                REGISTERED   ASTRACONNECTORID
STATUS
astra-connector     true         be475ae5-1511-4eaa-9b9e-712f09b0d065
Registered with Astra
```

記下 ASTRACONECTORID ；將叢集新增至 Astra Control 時、您將需要它。

接下來呢？

安裝 Astra Connector 之後、您就可以將私有叢集新增至 Astra Control Service 。

- "將私有提供者管理的叢集新增至 Astra Control Service"：請使用以下步驟來新增具有私有 IP 位址且由雲端供應商管理的叢集。您需要雲端供應商的服務主體帳戶、服務帳戶或使用者帳戶。
- "將私有自我管理叢集新增至 Astra Control Service"：請使用以下步驟來新增具有私有 IP 位址且由組織管理的叢集。您需要為想要新增的叢集建立一個 kubeconfig 檔案。

以取得更多資訊

- "新增叢集"

新增由供應商管理的叢集

將公有供應商託管的叢集新增至 **Astra Control Service**

在您設定雲端環境之後、您就可以建立 Kubernetes 叢集、然後將其新增至 Astra Control Service 。

- 建立Kubernetes叢集
- 將叢集新增至 Astra Control Service
- [變更預設儲存類別]

建立**Kubernetes**叢集

如果您還沒有叢集、可以建立符合的叢集 "Amazon Elastic Kubernetes服務（EKS）的Astra控制服務要求"。如果您還沒有叢集、可以建立符合的叢集 "Google Kubernetes Engine（GKE）的Astra控制服務要求"。如果您還沒有叢集、可以建立符合的叢集 "Azure Kubernetes Service（含Azure NetApp Files ）的Astra Control Service要求" 或 "Azure Kubernetes Service（Astra Control Service）與Azure託管磁碟的相關要求"。

> Astra Control Service支援使用Azure Active Directory（Azure AD）進行驗證與身分識別管理的高峰叢集。建立叢集時、請遵循中的指示 "正式文件" 設定叢集使用Azure AD。您必須確保叢集符合高峰管理Azure AD整合的要求。

將叢集新增至 **Astra Control Service**

登入Astra Control Service之後、您的第一步就是開始管理叢集。將叢集新增至Astra Control Service之前、您必須執行特定工作、並確保叢集符合特定需求。

當您管理 Azure Kubernetes Service 和 Google Kubernetes Engine 叢集時、請注意 Astra Control Provisioner 的安裝和生命週期管理有兩個選項：

- 您可以使用 Astra Control Service 來自動管理 Astra Control Provisioner 的生命週期。為達成此目的、請確定未安裝 Astra Trident 、且 Astra Control Provisioner 未在您要使用 Astra Control Service 管理的叢集上啟用。在這種情況下、 Astra Control Service 會在您開始管理叢集時自動啟用 Astra Control Provisioner 、並自動處理 Astra Control Provisioner 升級。
- 您可以自行管理 Astra Control 資源配置程式的生命週期。若要執行此作業、請先在叢集上啟用 Astra

Control Provisioner 、然後再使用 Astra Control Service 管理叢集。在這種情況下、 Astra Control Service 會偵測到 Astra Control Provisioner 已啟用、而且不會重新安裝或管理 Astra Control Provisioner 升級。請參閱 "啟用 Astra Control Provisioner" 針對步驟、請啟用 Astra Control Provisioner 。

當您使用 Astra Control Service 管理 Amazon Web Services 叢集時、如果您需要的儲存後端只能與 Astra Control Provisioner 搭配使用、則必須先在叢集上手動啟用 Astra Control Provisioner 、然後才能使用 Astra Control Service 進行管理。請參閱 "啟用 Astra Control Provisioner" 適用於啟用 Astra Control Provisioner 的步驟。

開始之前

Amazon Web Services

- 您應該擁有Json檔案、其中包含建立叢集的IAM使用者認證。 "瞭解如何建立IAM使用者"。
- Amazon FSX for NetApp ONTAP 需要 Astra Control Provisioner 。如果您打算將 Amazon FSX for NetApp ONTAP 作為 EKS 叢集的儲存後端、請參閱中的 Astra Control Provisioner 資訊 "EKS叢集需求"。
- （選用）如果您需要提供 `kubectl` 叢集的命令存取功能可讓其他不是叢集建立者的IAM使用者存取、請參閱中的指示 "如何在Amazon EKS中建立叢集後、提供其他IAM使用者和角色的存取權限？"。
- 如果您計畫將NetApp Cloud Volumes ONTAP 支援作為儲存後端、則需要設定Cloud Volumes ONTAP 支援以搭配Amazon Web Services使用的功能。請參閱Cloud Volumes ONTAP 《The》 "設定文件"。

步驟

1. （選用）如果您要新增 Amazon EKS 叢集、或想要自行管理 Astra Control Provisioner 的安裝與升級、請在叢集上啟用 Astra Control Provisioner 。請參閱 "啟用 Astra Control Provisioner" 以瞭解啟用步驟。
2. 在瀏覽器中開啟 Astra Control Service Web UI 。
3. 在儀表板上、選取*管理Kubernetes叢集*。

   依照提示新增叢集。

4. 供應商：選取您的雲端供應商、然後提供必要的認證資料以建立新的雲端執行個體、或選取要使用的現有雲端執行個體。
5. * Amazon Web Services *：上傳Json檔案或從剪貼簿貼上Json檔案的內容、以提供Amazon Web Services IAM使用者帳戶的詳細資料。

   Json檔案應包含建立叢集的IAM使用者認證。

6. * 其他 * ：此索引標籤僅適用於自我管理叢集。
   a. * Cloud 執行個體名稱 * ：為新增叢集時將建立的新雲端執行個體提供名稱。深入瞭解 "雲端執行個體"。
   b. 選擇*下一步*。

      Astra Control Service 會顯示您可以選擇的叢集清單。

   c. * 叢集 * ：從清單中選取要新增至 Astra Control Service 的叢集。

      > 當您從叢集清單中選取時、請注意 **Eligiblity** 欄。如果叢集為「不合格」或「部分合格」、請將游標移至狀態上方、以判斷叢集是否有問題。例如、它可能會識別叢集沒有工作節點。

d. 選擇*下一步*。

e. （可選） * Storage* ：（可選）選擇您希望 Kubernetes 應用程式部署到此叢集的儲存類別、以供預設使用。

7. 若要為叢集選取新的預設儲存類別、請啟用 * 指派新的預設儲存類別 * 核取方塊。
8. 從清單中選取新的預設儲存類別。

> 每個雲端供應商的儲存服務都會顯示下列價格、效能和恢復能力資訊：
>
> - Amazon Elastic Block Store：沒有可用的價格、效能或恢復能力資訊
> - Amazon FSX for NetApp ONTAP 不提供價格、效能或恢復能力資訊
> - NetApp Cloud Volumes ONTAP 產品：不提供價格、效能或恢復能力資訊

每個儲存類別都可以使用下列其中一項服務：

- "Amazon彈性區塊存放區"
- "Amazon FSX for NetApp ONTAP 產品"
- "NetApp Cloud Volumes ONTAP"

深入瞭解 "Amazon Web Services叢集的儲存類別"。深入瞭解 "適用於高效能叢集的儲存類別"。深入瞭解 "GKE叢集的儲存類別"。

a. 選擇*下一步*。

b. * 審查與核准 * ：檢閱組態詳細資料。

c. 選取 * 新增 * 將叢集新增至 Astra Control Service 。

結果

如果這是您為此雲端供應商新增的第一個叢集、 Astra Control Service 會為雲端供應商建立物件儲存區、以便備份在合格叢集上執行的應用程式。（當您新增此雲端供應商的後續叢集時、不會再建立其他物件存放區。）如果您指定預設儲存類別、Astra Control Service會設定您指定的預設儲存類別。對於在Amazon Web Services或Google Cloud Platform中管理的叢集、Astra Control Service也會在叢集上建立管理員帳戶。這些動作可能需要幾分鐘的時間。

變更預設儲存類別

您可以變更叢集的預設儲存類別。

使用**Astra Control**變更預設儲存類別

您可以從Astra Control中變更叢集的預設儲存類別。如果叢集使用先前安裝的儲存後端服務、您可能無法使用此方法來變更預設儲存類別（*設為預設*動作無法選取）。在這種情況下、您可以 [使用命令列變更預設儲存類別]。

步驟

1. 在Astra Control Service UI中、選取* Clusters*。
2. 在「叢集」頁面上、選取您要變更的叢集。
3. 選擇* Storage*（儲存設備）選項卡。

4. 選擇*儲存類別*類別。
5. 針對您要設為預設的儲存類別、選取「動作」功能表。
6. 選擇*設為預設*。

使用命令列變更預設儲存類別

您可以使用Kubernetes命令變更叢集的預設儲存類別。無論叢集的組態為何、此方法都能正常運作。

步驟

1. 登入Kubernetes叢集。
2. 列出叢集中的儲存類別：

```
kubectl get storageclass
```

3. 從預設儲存類別中移除預設指定。以<SC_NAME> 儲存類別的名稱取代支援：

```
kubectl patch storageclass <SC_NAME> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-
class":"false"}}}'
```

4. 將不同的儲存類別標示為預設。以<SC_NAME> 儲存類別的名稱取代支援：

```
kubectl patch storageclass <SC_NAME> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

5. 確認新的預設儲存類別：

```
kubectl get storageclass
```